

8-hybrid-model

June 19, 2026

1 Hybrid Model (MLP + CNN residual)

- MLP learns global physics
- CNN learns leftover spatial structure (residual correction)

The prediction is decomposed into two complementary components:

$$\hat{y} = \hat{y}_{\text{mlp}} + \hat{y}_{\text{cnn}}$$

Component	Role	Input Features
MLP	Captures global signal	V, Fr, Coverage, Color, dt2
CNN	Learns spatial correction	Nozzle axis (1120 positions)

The MLP handles the dominant global dependencies, while the CNN models residual spatial patterns that the MLP misses.

```
[ ]: import polars as pl
import matplotlib as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.model_selection import train_test_split

PARQUET_PATH = "../../../data/interim/encoded-collapsed-waveform.parquet"

df = pl.read_parquet(PARQUET_PATH)
```

```
[ ]: feature_cols = [c for c in df.columns if not c.startswith("Value")]
value_cols = [c for c in df.columns if c.startswith("Value_")]

X = df[feature_cols].to_numpy()
y = df[value_cols].to_numpy()

print("df shape:", df.shape)
print("X shape:", X.shape)
print("y shape:", y.shape)
print("value_cols:", len(value_cols))
```

```
print(df[value_cols].shape)
```

1.1 First Model

```
[ ]: X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32)

X_val = torch.tensor(X_val, dtype=torch.float32)
y_val = torch.tensor(y_val, dtype=torch.float32)

X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32)

print(type(X_train), X_train.shape)
print(type(y_train), y_train.shape)
print(type(X_val), X_val.shape)
print(type(y_val), y_val.shape)
```

```
[ ]: class HybridMLP_CNN(nn.Module):
    def __init__(self, input_dim, output_dim=1120):
        super().__init__()

        # --- MLP branch ---
        self.mlp = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim)
        )

        # --- CNN residual branch ---
        self.cnn_pre = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim)
        )

        self.cnn = nn.Sequential(
```

```

        nn.Conv1d(1, 16, kernel_size=5, padding=2),
        nn.ReLU(),
        nn.Conv1d(16, 1, kernel_size=5, padding=2)
    )

    def forward(self, x):
        # MLP global prediction
        y_mlp = self.mlp(x)

        # CNN residual path
        r = self.cnn_pre(x)                # (B, 1120)
        r = r.unsqueeze(1)                  # (B, 1, 1120)
        r = self.cnn(r).squeeze(1)         # (B, 1120)

        # final prediction
        return y_mlp + r

```

```

[ ]: model = HybridMLP_CNN(X.shape[1])

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

```

[ ]: X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32)

X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32)

for epoch in range(30):
    model.train()

    preds = model(X_train)
    loss = criterion(preds, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 5 == 0:
        model.eval()

        with torch.no_grad():
            val_preds = model(X_val)
            val_mse = F.mse_loss(val_preds, y_val).item()
            val_mae = F.l1_loss(val_preds, y_val).item()

```

```

        test_preds = model(X_test)
        test_loss = criterion(test_preds, y_test).item()

    torch.save({
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "loss": loss.item(),
    }, "hybrid_checkpoint.pth")

    print(
        f"Epoch {epoch} | "
        f"train {loss.item():.4f} | "
        f"val MSE {val_mse:.4f} | "
        f"test {test_loss:.4f}"
    )

```

1.2 Second Model

```

[ ]: import torch
import torch.nn as nn
import torch.nn.functional as F

class HybridMLP_CNN(nn.Module):
    def __init__(self, input_dim, output_dim=1120):
        super().__init__()

        # --- MLP branch: global prediction ---
        self.mlp = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim)
        )

        # --- CNN residual branch ---
        # Projects input to spatial representation, then refines with conv
        self.cnn_pre = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim)
        )

        self.cnn = nn.Sequential(
            # Layer 1: fine local structure (kernel 5, ~5 nozzles)
            nn.Conv1d(1, 16, kernel_size=5, padding=2),
            nn.BatchNorm1d(16),

```

```

        nn.ReLU(),
        # Layer 2: medium range (kernel 11, ~11 nozzles)
        nn.Conv1d(16, 32, kernel_size=11, padding=5),
        nn.BatchNorm1d(32),
        nn.ReLU(),
        # Layer 3: broader neighbourhood (kernel 21, ~21 nozzles)
        nn.Conv1d(32, 16, kernel_size=21, padding=10),
        nn.BatchNorm1d(16),
        nn.ReLU(),
        # Project back to 1 channel
        nn.Conv1d(16, 1, kernel_size=5, padding=2)
    )

    def forward(self, x):
        # MLP global prediction
        y_mlp = self.mlp(x)

        # CNN spatial residual path
        r = self.cnn_pre(x)          # (B, 1120)
        r = r.unsqueeze(1)           # (B, 1, 1120)
        r = self.cnn(r).squeeze(1)   # (B, 1120)

        return y_mlp + r

```

```

[ ]: model = HybridMLP_CNN(X_train.shape[1])
criterion = nn.L1Loss() # MAE directly - consistent with your 0.018 baseline
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=10, verbose=True
)

```

```

[ ]: from torch.utils.data import DataLoader, TensorDataset

train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")

X_val = torch.tensor(X_test, dtype=torch.float32) # CPU
y_val = torch.tensor(y_test, dtype=torch.float32) # CPU

model = model.to(device)

for epoch in range(200):
    model.train()
    train_loss_full = 0
    train_mae = 0

```

```

for X_batch, y_batch in train_loader:  # ← iterate batches
    pred = model(X_batch)
    loss = criterion(pred, y_batch)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    train_loss_full += loss.item()
    train_mae += F.l1_loss(pred, y_batch).item()

train_loss_full /= len(train_loader)
train_mae       /= len(train_loader)

model.eval()
with torch.no_grad():
    pred_val = model(X_val.to(device))
    val_loss = criterion(pred_val, y_val.to(device)).item()
    test_mae = F.l1_loss(pred_val, y_val.to(device)).item()

scheduler.step(val_loss)

if epoch % 10 == 0:
    print(epoch, train_loss_full, val_loss, train_mae, test_mae)

```

1.3 New Data Split (1st Model)

```
[ ]: model = HybridMLP_CNN(X.shape[1])
```

```

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

```

[ ]: # trying with new data splits
configs = df.select(['V', 'F_r']).unique()
train_configs, test_configs = train_test_split(configs.to_pandas(), test_size=0.
    ↪2, random_state=42)

# Convert back to sets for filtering
train_set = set(zip(train_configs['V'], train_configs['F_r']))

train_mask = df.select(['V', 'F_r']).map_rows(lambda r: r in train_set).
    ↪to_series()

X_train = df.filter(train_mask)[feature_cols].to_numpy()
X_test  = df.filter(~train_mask)[feature_cols].to_numpy()

```

```
y_train = df.filter(train_mask)[value_cols].to_numpy()
y_test = df.filter(~train_mask)[value_cols].to_numpy()
```

```
[ ]: X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32)

X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32)

for epoch in range(200):
    model.train()

    preds = model(X_train_t)
    loss = criterion(preds, y_train_t)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 10 == 0:
        torch.save({
            "epoch": epoch,
            "model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict(),
            "loss": loss.item(),
        }, "new_split_checkpoint.pth")
        model.eval()
        with torch.no_grad():
            train_preds = model(X_train_t)
            train_loss_full = criterion(train_preds, y_train_t).item()
            train_mae = torch.mean(torch.abs(train_preds - y_train_t)).item()

            test_preds = model(X_test_t)
            test_loss = criterion(test_preds, y_test_t).item()
            test_mae = torch.mean(torch.abs(test_preds - y_test_t)).item()

        print(epoch, train_loss_full, test_loss, train_mae, test_mae)
```

1.3.1 Notes

To incorporate the spatial structure of the printhead, a hybrid MLP-CNN architecture was developed.

The model consists of two branches: an MLP that produces a global prediction across all 1,120 nozzle outputs, and a CNN residual branch that applies learned local filters over the nozzle axis to correct spatially correlated errors. The intuition is that neighbouring nozzles are physically coupled and their outputs are not independent, so convolutional kernels with receptive fields of 5

to 21 nozzles can capture local interaction patterns that a fully connected layer cannot.

The split is performed on unique (V, F_r) operating point combinations. All unique configurations are extracted first, then split 80/20 into train and test groups. A boolean mask is then applied to the full dataset, ensuring every row belonging to the same operating point lands on the same side of the split.

This guarantees that the model is evaluated on configurations it has never seen during training — if a (V, F_r) pair is in the test set, none of its rows appear in training. This is called a group split, and it tests genuine generalisation to unseen operating conditions rather than interpolation within already-seen configurations, which a random row-level split would allow.

After 190 epochs the model achieved a test MAE of 0.0249, compared to 0.139 for the plain MLP baseline, representing an 82% reduction in error. The model continued improving gradually through the full training run without fully plateauing, suggesting further gains may be possible with additional training or architectural refinements. While this does not yet match the XGBoost + PCA baseline of 0.018, the result is obtained with a principled split strategy and an architecture that explicitly models the spatial organisation of the printhead output, making it a more interpretable and physically motivated approach.

```
[ ]: for i in [0, 5, 10, 20]:
    sample = torch.tensor(X_test[i:i+1], dtype=torch.float32).to(device)
    with torch.no_grad():
        pred = model(sample).squeeze().cpu().numpy()
    plt.plot(y_test[i], label='actual')
    plt.plot(pred, label='predicted')
    plt.title(f'sample {i}')
    plt.show()
```

```
[ ]: print(y_test.mean(), y_test.std(), y_test.min(), y_test.max())
    print(y_train.mean(), y_train.std(), y_train.min(), y_train.max())
```

2 Conclusion

We started by treating each row as an independent sample with global features (Color, Coverage, V, Fr, dt2) and a flattened 1120-nozzle output per chip, using one-hot encoding and standard regression models (MLP first, then CNN, then a hybrid).

The MLP already performed reasonably well, while the CNN and hybrid only gave marginal improvements, which indicated that the dominant signal was global process conditions rather than strong local nozzle-to-nozzle structure. We then tried to impose spatial structure by reshaping outputs into chip-based tensors and exploring more structured splits across experimental conditions, but this exposed a deeper issue: the dataset is not a complete, uniform 30-chip grid per condition, with some conditions missing chips (22–30 instead of always 30), which breaks strict spatial assumptions.

As a result, the CNN was applied along the wrong axis (nozzle index instead of chip index), and the assumed full spatial continuity across the printhead was only partially valid.

Moving forward, the plan is to stop forcing full-grid convolutional assumptions and instead explicitly

model the true hierarchy: each chip is treated as a structured unit (1120-dimensional signal), and the printhead becomes an ordered sequence of up to 30 chips with missing elements.

The next step is to build a chip-level encoder and then apply a sequence model over the chip axis (with masking for missing chips) so that spatial relationships between adjacent chips are learned directly, rather than being implicitly assumed or destroyed by flattening.